

Edumerge-360: Enterprise Campus Operating System

Unified 5-in-1 Production Architecture for Higher Education Institutions

Candidate: Mahesh Mekala	Target Role: Product Engineer / SDE
GitHub: https://github.com/Maheshmekala	Submission Email: tech_interview@edumerge.com
Repository: https://github.com/Maheshmekala/edumerge-360	Date: September 25, 2026

1. Architectural Scope & Problem Understanding

LIVE REVIEWER ACCESS & INSTANT EVALUATION LINKS

- Live Public URL (Zero-Setup): <https://roles-edit-pregnant-hamilton.trycloudflare.com>
- 1-Click Role Switcher: Persistent dropdown in header allows instant switching between Super Admin, Dean, Finance, Faculty, Counsellor, and Student personas without repeated manual login/logout.

2. The 5 Integrated Business Engines at a Glance

Module 1: Smart Attendance
Module 2: Fee Collection & 3-Way Rec
Module 3: Intelligent Timetable CSP
Module 4: Student Support & SLAs
Module 5: Admission Lead CRM

Engineering Decisions, Edge Cases & Defensibility

3. Deep Technical Implementation & Edge-Case Safeguards

A. Strict Integer Minor Currency & Zero Floating-Point Drift (Fee Engine)

B. Algorithmic 3-Way Bank Statement Matcher (Reconciliation Engine)

Matches ingested Bank Settlement CSV lines against internal ERP Payments and Invoices across 4 passes: Pass 1 matches exact UTR, date, and cents; Pass 2 detects intermediary wire deductions (e.g. \$15 fee subtracted by clearing network); Pass 3 flags unrecognized direct inward credits; Pass 4 isolates delayed bank settlements. All run in $O(N)$ time with zero manual ledger reconciliation.

C. Constraint Satisfaction Problem (CSP) Solver with MRV Heuristic (Timetable)

D. Pessimistic Date-Locking & Maker-Checker Workflows (Attendance Engine)

Regulatory accreditation audits fail when faculty backdate attendance weeks later. The attendance API strictly checks `sessionDate < today` and returns HTTP 423 Locked. Corrections must follow a formal maker-checker flow: faculty provide written justification, and only the Academic Dean can review and atomically apply updates with full audit logging.

E. Weighted Round-Robin Routing & SLA Urgency Timers (CRM & Support)

Inbound admission leads are assigned using a weighted algorithm prioritizing course specialists with available capacity while balancing active lead counts across counsellors to eliminate hoarding. Support tickets feature real-time SLA countdown timers that visually transition urgency (Normal -> High -> Critical -> Breached) with 1-click managerial escalation.

4. Architectural Trade-offs & Assumptions

? Modular Monolith vs Microservices: For an educational campus of 5,000 students and 200 faculty, microservices introduce distributed transaction overhead (2PC/Sagas) and latency. A modular monolith using Next.js 15 App Router with transactional business engines ensures atomic ACID guarantees across invoices, attendance, and timetables.

? SQLite / PostgreSQL Portability: Developed with zero-dependency SQLite for instantaneous local evaluation, with a schema 100% compatible with production PostgreSQL and Docker deployment.

Mandatory AI Usage & Engineering Validation Report

Compliance with Pre-Drive Assignment Brief Page 6 Requirements

AI TOOL USED:	Claude (Anthropic) & ChatGPT (GPT-4o)
Role in Workflow:	Rapid scaffolding, boilerplate generation, and schema brainstorming.

WHAT I ASKED AI TO DO:

- 1. Brainstorm relational schema links connecting the 5 domains into an integrated enterprise ERP.
- 2. Scaffold boilerplate Next.js App Router API route handlers and initial Tailwind layouts.
- 3. Draft an initial recursive backtracking skeleton for academic timetable scheduling.
- 4. Generate realistic synthetic institutional seed data (courses, subjects, faculty, and bank CSV lines).

PROMPT THAT WAS MOST USEFUL:

"Implement a pure TypeScript Constraint Satisfaction Problem (CSP) solver using backtracking and forward checking for college timetable scheduling. Hard constraints must include no professor double-booking, no classroom double-booking, room capacity >= class size, and lab subjects in lab rooms. If impossible, return structured diagnostics."

CODE GENERATED BY AI (What part?): Initial Prisma schema draft, recursive timetable skeleton, bank CSV interfaces.

CODE I MODIFIED (What part?):

- Converted all financial amounts to integer minor units (paise/cents) to eliminate float rounding errors.
- Wrapped fee payments, invoice balances, and head distributions in atomic prisma.\$transaction().
- Implemented hard backend date-locking check on attendance sessions returning HTTP 423 Locked.
- Engineered persistent evaluator 1-click role switcher for fast technical assessment.

AI OUTPUT THAT WAS WRONG / SUB-OPTIMAL:

HOW I IDENTIFIED THE PROBLEM: Automated Vitest tests (tests/timetable-csp.test.ts) failed with unhandled stack overflows.

HOW I FIXED IT:

- 1. Replaced random shuffle with formal Minimum Remaining Values (MRV) heuristic (scheduling lab subjects first).
- 2. Added instant Pre-Solve Feasibility Check: halts in <5ms with HOURS_EXCEED_SLOTS diagnostic if hours exceed periods.
- 3. Introduced deterministic tracking sets (facultyBusy, roomBusy, divisionGrid) to evaluate hard constraints in O(1) time.

AUTOMATED VALIDATION RESULTS: 17/17 TESTS PASSING IN VITEST
Lead Router (3/3) · 3-Way Reconciliation (4/4) · Attendance Radar (4/4) · SLA Engine (3/3) · Timetable CSP (3/3)